

Projet Python — Séance 3

Réponses aux questions : pandas, jointures et découverte de numpy

Partie A — Découvrir pandas

1. Qu'est-ce qu'un DataFrame ? Une Series ?

Un DataFrame est un tableau à deux dimensions, comme une feuille Excel : il a des lignes, repérées par un index (ici les dates), et des colonnes nommées, chacune pouvant avoir un type différent (dates, nombres décimaux, texte). Une Series est un tableau à une seule dimension : c'est une colonne, avec son index et un seul type de données.

Un DataFrame peut donc se voir comme un ensemble de Series qui partagent le même index. Quand on écrit `df[ 'taux' ]`, on obtient une Series.

2. Pourquoi utiliser pandas plutôt que des boucles et le module csv ? Que signifie « vectorisation » ?

Avec le module csv, on lit le fichier ligne par ligne et on obtient des listes de chaînes de caractères qu'il faut convertir soi-même en nombres ou en dates, puis on écrit une boucle pour chaque calcul (moyenne, minimum, filtre...). Avec pandas, `read_csv` charge tout le fichier en une seule ligne, devine les types, sait lire les dates et gère les valeurs manquantes. Des méthodes toutes prêtes (`describe`, `mean`, `resample`, `merge`...) remplacent les boucles. Le code est plus court, plus lisible, moins sujet aux erreurs et beaucoup plus rapide.

La vectorisation consiste à appliquer une opération à toute une colonne (ou à tout un tableau) en une seule instruction, par exemple `df[ 'taux' ] * 100`, au lieu d'écrire une boucle `for` sur chaque valeur. La boucle est alors exécutée dans du code compilé (en C) à l'intérieur de pandas et numpy, et non par l'interpréteur Python, ce qui est bien plus rapide.

3. Pourquoi déconseille-t-on `iterrows()` ?

`iterrows()` parcourt le DataFrame ligne par ligne et, à chaque tour, construit une Series pour représenter la ligne. C'est donc une boucle Python classique, lente : fabriquer un objet par ligne coûte cher, surtout sur de gros fichiers. De plus, une Series n'a qu'un seul type : pandas convertit toutes les valeurs de la ligne vers un type commun. Si la ligne mélange entiers et décimaux, les entiers deviennent des décimaux, et si elle contient du texte, tout devient de type `object`, ce qui fait perdre les types d'origine.

On préfère donc les opérations vectorisées. À défaut, `itertuples()` est plus rapide et conserve mieux les types, et `apply` est une autre option.

4. Comment pandas représente-t-il les valeurs manquantes (NaN) ? Conséquences sur les moyennes ?

pandas représente une valeur manquante par NaN (Not a Number), un nombre décimal spécial hérité de numpy (`np.nan`). On les repère avec `isna()` (on ne peut pas écrire `x == NaN`, car NaN n'est égal à rien, pas même à lui-même) et on les traite avec `dropna()` (supprimer les lignes concernées) ou `fillna()` (les remplacer).

Conséquences sur les moyennes : par défaut, les méthodes de pandas (`mean`, `sum`, `min`, `max`) ignorent les NaN (paramètre `skipna=True`). La moyenne est donc calculée seulement sur les valeurs présentes, c'est-à-dire sur moins de valeurs que la taille de la colonne. Si les valeurs

manquantes ne sont pas « neutres » (par exemple des jours particuliers), la moyenne peut être biaisée. À l'inverse, `np.mean` de numpy renvoie NaN dès qu'il y a un NaN dans les données. Enfin, un NaN se propage dans les calculs (`NaN + 3` donne NaN), et une colonne d'entiers qui contient un NaN est convertie en décimaux.

Partie B — Nettoyer, consolider et analyser

Justification du choix du forward-fill (B.1)

Les taux ne sont pas publiés les jours fériés. Ces jours-là, le taux de référence en vigueur reste le dernier taux publié : le forward-fill est donc cohérent avec la réalité. Il n'utilise que des informations passées. Une interpolation linéaire, elle, utiliserait la valeur du lendemain, qu'on ne connaissait pas à la date concernée. Supprimer les jours manquants laisserait un calendrier irrégulier, ce qui fausserait la moyenne mobile (définie en nombre de jours ouvrés) et compliquerait les jointures entre deux devises. Réindexer sur un calendrier continu de jours ouvrés puis propager la dernière valeur donne une série complète et sans NaN.

Questions

1. C'est quoi le forward-fill ? Quelle hypothèse fait-on en l'utilisant ?

Le forward-fill (`ffill`, « remplissage vers l'avant ») remplace chaque valeur manquante par la dernière valeur connue qui la précède dans la série. Par exemple, si le taux du jeudi est de 1,08 et que celui du vendredi est manquant, le vendredi reçoit 1,08.

L'hypothèse faite est que la valeur reste inchangée jusqu'à la prochaine publication, c'est-à-dire qu'aucune information nouvelle n'est arrivée pendant le trou. C'est raisonnable pour un jour férié, moins pour un long trou. Le forward-fill a un avantage, il n'utilise que le passé (pas de biais d'anticipation). Il a aussi des limites : il crée des variations artificiellement nulles, ce qui peut sous-estimer la volatilité, et il ne peut pas remplir un trou situé au tout début de la série.

2. À quoi sert `resample` ? Que signifie une moyenne mobile ?

`resample` change la fréquence d'une série temporelle : on regroupe les observations par période (par exemple par mois), puis on résume chaque groupe avec une fonction (`mean`, `last`, `min`, `max`, `sum`...). L'index doit être de type date. Cela sert par exemple à passer de taux journaliers à un taux moyen mensuel, pour réduire le bruit, pour comparer avec des données de fréquence plus faible ou pour synthétiser une longue période.

Une moyenne mobile est la moyenne des `n` dernières valeurs, calculée avec une fenêtre qui glisse d'un jour à chaque fois. La moyenne mobile sur 5 jours du mardi est la moyenne des 5 derniers jours ouvrés jusqu'à mardi. Elle lisse les fluctuations de court terme pour faire ressortir la tendance, mais elle réagit avec un certain retard, et ses `n - 1` premières valeurs sont NaN, car la fenêtre n'est pas encore pleine. La différence avec `resample` est que `resample` réduit le nombre de lignes (une par période), alors que la moyenne mobile garde une valeur par jour, avec des fenêtres qui se chevauchent.

Partie C — Les jointures

1. Rappelez les quatre types de jointure (inner, left, right, outer) : que garde chacune ?

`inner` ne garde que les clés présentes dans les deux tables (l'intersection). `left` garde toutes les lignes de la table de gauche, et les colonnes de droite valent NaN là où il n'y a pas de correspondance. `right` garde toutes les lignes de la table de droite, et les colonnes de gauche valent NaN là où il n'y a pas de correspondance. `outer` garde toutes les lignes des deux tables (l'union), avec des NaN de chaque côté là où il manque une correspondance.

Avec une table de gauche de clés {1, 2} et une table de droite de clés {2, 3}, on obtient donc : `inner` {2}, `left` {1, 2}, `right` {2, 3} et `outer` {1, 2, 3}.

2. Sur quelle colonne (clé) joint-on ? Que devient une ligne sans correspondance dans l'autre table ?

On joint sur une colonne présente dans les deux tables, appelée la clé. Pandas associe les lignes qui ont la même valeur dans cette colonne. Pour nos taux de change, la clé est la date.

Le sort d'une ligne sans correspondance dépend du paramètre `how`. Avec `inner`, elle est supprimée. Avec `left`, `right` ou `outer`, elle est conservée si elle appartient à la table dont on garde toutes les lignes, et les colonnes de l'autre table sont alors remplies par NaN. Attention : si une clé apparaît plusieurs fois, la jointure produit toutes les combinaisons possibles (2 lignes à gauche et 3 à droite pour la même clé donnent 6 lignes), donc il faut toujours vérifier le nombre de lignes après un `merge`.

3. Pour réunir deux séries de taux sur la même période, quelle jointure choisir et pourquoi ?

On choisit une jointure `inner` sur la date. Sur la même période, les deux tables ont exactement les mêmes dates, donc les quatre types de jointure donnent le même résultat, sans aucun NaN. `inner` est le choix le plus sûr : il ne garde que les dates où les deux taux sont connus, donc chaque ligne est directement exploitable (calcul d'un taux croisé, d'une corrélation, etc.).

Si les calendriers de publication des deux devises différaient, `inner` ferait perdre des lignes. On utiliserait alors `outer`, suivi d'un forward-fill comme en B.1. Dans tous les cas, on vérifie le nombre de lignes après le `merge`.

4. Quelle différence entre `merge` et `concat` ? Dans quel cas utiliser l'un ou l'autre ?

`merge` joint deux tables en associant leurs lignes selon une clé, comme un `JOIN` en SQL : il ajoute des colonnes, en faisant correspondre les lignes qui ont la même clé (par exemple, le taux USD et le taux GBP de la même date sur une même ligne). `concat` n'apparie aucune clé : il empile les tables, soit les unes sous les autres (`axis=0`, par exemple les taux de janvier puis ceux de février, qui ont les mêmes colonnes), soit côte à côte (`axis=1`, en alignant simplement les lignes sur l'index).

On utilise `merge` quand il faut retrouver, pour chaque ligne, l'information correspondante d'une autre table grâce à une clé. On utilise `concat` pour ajouter des lignes de même nature ou pour assembler des tables déjà alignées. Quant à `join`, c'est un `merge` qui s'appuie sur l'index plutôt que sur une colonne.

Partie D — Un peu de numpy (et aperçu de polars)

1. Différence entre une liste Python et un tableau numpy (ndarray) ? C'est quoi un dtype ? Le broadcasting ?

Une liste Python peut contenir des éléments de types différents (entier, texte, autre liste...) : elle stocke des références vers des objets Python dispersés en mémoire, et les opérations arithmétiques ne s'appliquent pas terme à terme (`[1, 2] * 3` répète la liste et donne `[1, 2, 1, 2, 1, 2]`). Un tableau numpy ne contient que des éléments du même type, rangés côte à côte dans un bloc de mémoire continu, et les opérations s'appliquent élément par élément (`np.array([1, 2]) * 3` donne `[3, 6]`).

Le dtype est le type des éléments du tableau, identique pour tous : par exemple `float64` (décimaux sur 64 bits), `int64` (entiers) ou `datetime64`. Il détermine la place mémoire de chaque élément et les opérations possibles.

Le broadcasting est la règle qui permet d'effectuer une opération entre un tableau et un nombre (ou entre des tableaux de formes compatibles) sans écrire de boucle : le nombre est « étendu » virtuellement à la taille du tableau. Par exemple, `taux * 100` multiplie chaque taux par 100, et `taux - taux.mean()` retranche la moyenne à tous les éléments.

2. Pourquoi numpy est-il plus rapide que des boucles Python ?

Il y a trois raisons principales. D'abord, la boucle est exécutée dans du code compilé (C) et non par l'interpréteur Python. En Python pur, chaque tour de boucle vérifie le type des objets, appelle l'opération et crée un nouvel objet, ce qui coûte cher. Ensuite, les éléments d'un tableau numpy ont tous le même type et sont rangés de façon contiguë en mémoire : le processeur les lit efficacement et sait exactement comment les traiter, sans vérification élément par élément. Enfin, numpy s'appuie sur des instructions vectorielles du processeur (SIMD) et sur des bibliothèques optimisées qui traitent plusieurs valeurs à la fois. Une opération vectorisée est ainsi souvent des dizaines de fois plus rapide qu'une boucle Python équivalente.

3. Différences entre pandas et polars (langage d'implémentation, usage des cœurs, évaluation paresseuse) ? Quand choisir l'un ou l'autre ?

Langage d'implémentation. pandas est écrit en Python, avec des parties critiques en C et Cython, et repose sur numpy. Polars est écrit en Rust, un langage compilé, et utilise un format de mémoire en colonnes (Apache Arrow).

Usage des cœurs. pandas exécute l'essentiel de ses opérations sur un seul cœur du processeur. Polars parallélise automatiquement le travail sur tous les cœurs disponibles.

Évaluation paresseuse. pandas est en évaluation immédiate (eager) : chaque instruction est exécutée dès qu'elle est écrite. Polars propose en plus un mode paresseux (lazy) : on décrit d'abord toutes les opérations, puis Polars construit un plan de requête, l'optimise (par exemple en ne lisant que les colonnes utiles et en filtrant le plus tôt possible) et l'exécute seulement au moment de `collect()`.

Quand choisir l'un ou l'autre. pandas convient pour des données de taille petite ou moyenne, une exploration rapide et pour s'intégrer à un vaste écosystème (matplotlib, statsmodels, scikit-learn...), avec beaucoup de documentation. Polars est préférable pour de gros volumes de données ou quand la vitesse et la mémoire comptent, notamment pour des chaînes de traitement longues. Pour nos séries de taux, qui sont petites, la différence de temps sera faible.